

Тема урока: строки

Повторение материала

Строки в Python используются, когда надо работать с текстовыми данными.

Создание строки. Для создания строк мы используем парные кавычки `' '` или `" "`:

```
s1 = 'Python'
s2 = "Pascal"
```

Считывание строки. Для считывания текстовых данных в строковую переменную мы используем функцию `input()`:

```
s = input() # считали текст
num = int(input()) # считали текст и преобразовали его в целое число
```

Пустая строка. Для создания пустой строки мы пишем `s = ''` или `s = ""`. Пустая строка – это аналог числа `0`.

Длина строки. Для определения длины строки (количества символов) мы используем встроенную функцию `len()`:

```
s = 'Hello'
n = len(s) # значение переменной равно 5
print(n)
```

Конкатенация и умножение на число. Операторы `+` и `*` можно использовать для строк. Оператор `+` сцепляет две и более строк. Это называется конкатенацией строк. Оператор `*` повторяет строку указанное количество раз.

Выражение	Результат
<code>'AB' + 'cd'</code>	<code>'ABcd'</code>
<code>'A' + '7' + 'B'</code>	<code>'A7B'</code>
<code>'Hi' * 4</code>	<code>'HiHiHiHi'</code>

Оператор принадлежности `in`. С помощью оператора `in` мы можем проверять, находится ли одна строка в составе другой. То есть, является ли одна строка подстрокой другой:

```
s = 'All you need is love'
if 'love' in s:
    print('❤️')
else:
    print('💔')
```

Так как строка `s` содержит подстроку `'love'`, то будет выведен смайлик ❤️.



В Python можно использовать смайлики [emoji](#) 🍌

Индексация строк

Очень часто бывает необходимо обратиться к конкретному символу в строке. Для этого в Python используются квадратные скобки `[]`, в которых указывается индекс (номер) нужного символа в строке.

Пусть `s = 'Python'`. Таблица ниже показывает, как работает индексация:

Выражение	Результат	Пояснение
<code>s[0]</code>	P	первый символ строки
<code>s[1]</code>	y	второй символ строки
<code>s[2]</code>	t	третий символ строки
<code>s[3]</code>	h	четвертый символ строки
<code>s[4]</code>	o	пятый символ строки
<code>s[5]</code>	n	шестой символ строки



Обратите внимание, первый символ строки равен `s[0]`, а не `s[1]`. В Python индексация начинается с `0`, по аналогии с функцией `range(n)`, которая генерировала последовательность натуральных чисел от `0` до `n - 1`.

В отличие от многих языков программирования, в Python есть возможность работы с отрицательными индексами. Если первый символ строки имеет индекс `0`, то последнему элементу присваивается индекс `-1`.

Выражение	Результат	Пояснение
<code>s[-6]</code>	P	первый символ строки
<code>s[-5]</code>	y	второй символ строки
<code>s[-4]</code>	t	третий символ строки
<code>s[-3]</code>	h	четвертый символ строки
<code>s[-2]</code>	o	пятый символ строки
<code>s[-1]</code>	n	шестой символ строки

Таким образом, получаем

Положительные индексы	0	1	2	3	4	5
Строка	P	y	t	h	o	n
Отрицательные индексы	-6	-5	-4	-3	-2	-1

0	1	2	3	4	5
P	y	t	h	o	n
-6	-5	-4	-3	-2	-1

Частая ошибка у начинающих программистов — обращение по несуществующему индексу в строке.

Например, если `s = 'Python'`, и мы попытаемся обратиться к `s[17]`, то мы получим ошибку:

```
IndexError: string index out of range
```

Ошибка возникает, поскольку строка содержит всего 6 символов.



Обратите внимание: если длина строки `s` равна `len(s)`, то при положительной нумерации слева направо, последний элемент имеет индекс равный `len(s) - 1`, а при отрицательной индексации справа налево, первый элемент имеет индекс равный `-len(s)`.

Итерирование строк

Очень часто нужно просканировать всю строку целиком, обрабатывая каждый ее символ. Для этого удобно использовать цикл `for`. Напишем программу, которая выводит каждый символ строки на отдельной строке:

```
s = 'abcdef'
for i in range(len(s)):
    print(s[i])
```

Результатом выполнения такой программы будут строки:

```
a
b
c
d
e
f
```

Мы передаем в функцию `range()` длину строки `len(s)`. В нашем случае длина строки `s`, равна 6. Таким образом, вызов функции `range(len(s))` имеет вид `range(6)` и переменная цикла `i` последовательно перебирает все значения от 0 до 5. Это означает, что выражение `s[i]` последовательно вернет все символы строки `s`. Такой способ итерации строки удобен, когда нам нужен не только сам элемент `s[i]`, но и его индекс `i`.

Если нам не нужен индекс самого символа, то мы можем использовать более короткий способ итерации:

```
s = 'abcdef'
for c in s:
    print(c)
```

Этот цикл пройдет по строке `s`, придавая переменной цикла `c` значение каждого символа (!) в отличие от предыдущего цикла, в котором переменная цикла «бегала» по индексам строки.



Обратите внимание на обозначение переменных цикла. В первом цикле мы используем имя `i`, что соответствует стандартной идеологии наименования переменных цикла. Во втором цикле, мы назвали переменную буквой `c` – первая буква слова char (символ).

Срезы строк

В предыдущем уроке мы научились работать с конкретными символами строки с помощью индексов `[]`. Иногда нужно бывает работать с целыми частями строки, в таком случае мы используем **срезы (slices)**. Срезы похожи на комбинацию индексации и функции `range()`.

Рассмотрим строку `s = 'abcdefghij'` .

Положительные индексы	0	1	2	3	4	5	6	7	8	9
Строка	a	b	c	d	e	f	g	h	i	j
Отрицательные индексы	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

С помощью среза мы можем получить несколько символов исходной строки, создав диапазон индексов разделенных двоеточием `s[x:y]` .

Приведённый ниже код:

```
print(s[2:5])
print(s[0:6])
print(s[2:7])
```

ВЫВОДИТ:

```
cde
abcdef
cdefg
```

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

При построении среза `s[x:y]` первое число – это то место, где начинается срез (**включительно**), а второе – это место, где заканчивается срез (**невключительно**). Разрезая строки, мы создаем подстроку, которая по сути является строкой внутри другой строки.

Срез до конца, от начала

Если опустить второй параметр в срезе `s[x:]` (но поставить двоеточие), то срез берется до конца строки. Аналогично если опустить первый параметр `s[:y]`, то можно взять срез от начала строки. Срез `s[:]` совпадает с самой строкой `s`.

Приведённый ниже код:

```
print(s[2:])  
print(s[:7])
```

ВЫВОДИТ:

```
cdefghij  
abcdefg
```



Срез `s[:]` возвращает исходную строку.

Отрицательные индексы в срезе

Мы также можем использовать отрицательные индексы для создания срезов. Как уже говорилось ранее, отрицательные индексы строки начинаются с `-1` и отсчитываются до достижения начала строки. При использовании отрицательных индексов **первый параметр среза должен быть меньше второго, либо должен быть пропущен**.

Приведённый ниже код:

```
print(s[-9:-4])
print(s[-3:])
print(s[:-3])
```

ВЫВОДИТ:

```
bcdef
hij
abcdefg
```

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1



Удалить из строки последний символ можно при помощи среза `s[:-1]`.

Шаг среза

Мы можем передать в срез третий необязательный параметр, который отвечает за шаг среза. К примеру, срез `s[1:7:2]` создаст строку `bdf` состоящую из каждого второго символа (индексы `1, 3, 5`, правая граница не включена в срез).

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Отрицательный шаг среза

Если в качестве шага среза указать **отрицательное число**, то символы будут идти в обратном порядке.

Приведённый ниже код:

```
print(s[::-1])
```

ВЫВОДИТ:

```
jihgfedcba
```

Приведённый ниже код:

```
print(s[1:7:2])
print(s[3::2])
print(s[:7:3])
print(s[::-2])
print(s[::-1])
print(s[::-2])
```

ВЫВОДИТ:

```
bdf
dfhj
adg
acegi
jihgfedcba
jhfdb
```

Подводя итог

```
s = 'abcdefghij'
```

Программный код	Результат	Пояснение
<code>s[2:5]</code>	<code>cde</code>	строка состоящая из символов с индексами 2, 3, 4
<code>s[:5]</code>	<code>abcde</code>	первые пять символов строки
<code>s[5:]</code>	<code>fghij</code>	строка состоящая из символов с индексами от 5 до конца
<code>s[-2:]</code>	<code>ij</code>	последние два символа строки
<code>s[:]</code>	<code>abcdefghij</code>	вся строка целиком
<code>s[1:7:2]</code>	<code>bdf</code>	строка, состоящая из каждого второго символа с индексами от 1 до 6
<code>s[::-1]</code>	<code>jihgfedcba</code>	строка в обратном порядке, так как шаг отрицательный

Изменение символа строки по индексу

Предположим, у нас есть строка `s = 'abcdefghij'` и мы хотим заменить символ с индексом 4 на `'X'`. Можно попытаться написать код:

```
s[4] = 'X'
```

Однако такой код не работает. В Python строки являются **неизменяемыми**, то есть мы не можем менять их содержимое с помощью индексатора.

Если мы хотим поменять какой-либо символ строки `s`, мы должны создать новую строку. Следующий код использует срезы и решает поставленную задачу:

```
s = s[:4] + 'X' + s[5:]
```

Мы создаем два среза: от начала строки до 3 символа и с 5 символа по конец строки, а между ними вставляем нужный нам символ, который встанет на 4 позицию.

Примечания

Примечание 1. Синтаксис срезов строк очень похож на синтаксис функции `range()`.

Примечание 2. Если первый параметр среза больше второго, то срез создает пустую строку.

Методы и функции

Мы уже знакомы с некоторыми встроенными функциями:

`min()`, `max()`, `len()`, `int()`, `float()` и так далее. Метод – специализированная функция, тесно связанная с объектом. Как и функция, метод вызывается для выполнения отдельной задачи, но он вызывается для определенного объекта и “знает” о своем целевом объекте во время выполнения.

Таким образом: метод – функция, применяемая к объекту. В данном случае к строке. Метод вызывается в виде `имя_объекта.имя_метода(параметры)`.

Например, `s.find('e')` – это применение к строке `s` метода `find` с одним параметром `'e'`.

Методы строкового типа данных можно разделить на три группы:

1. **Конвертация регистра;**
2. Поиск и замена;
3. Классификация символов.

Конвертация регистра

Методы в этой группе выполняют преобразование регистра для строк.

Метод `capitalize()`

Метод `capitalize()` возвращает копию строки `s`, в которой первый символ имеет верхний регистр, а все остальные символы имеют нижний регистр.

Приведённый ниже код:

```
s = 'fo0 BaR BAZ quX'  
print(s.capitalize())
```

ВЫВОДИТ:

```
Foo bar baz qux
```

Символы, не являющиеся буквами алфавита, остаются неизменными.

Приведённый ниже код:

```
s = 'foo123#BAR#.'  
print(s.capitalize())
```

ВЫВОДИТ:

```
Foo123#bar#.
```

Метод `swapcase()`

Метод `swapcase()` возвращает копию строки `s`, в которой все символы, имеющие верхний регистр, преобразуются в символы нижнего регистра и наоборот.

Приведённый ниже код:

```
s = 'F00 Bar 123 baz qUX'  
print(s.swapcase())
```

ВЫВОДИТ:

```
foo bAR 123 BAZ QuX
```

Метод `title()`

Метод `title()` возвращает копию строки `s`, в которой первый символ каждого слова переводится в верхний регистр.

Приведённый ниже код:

```
s = 'the sun also rises'  
print(s.title())
```

ВЫВОДИТ:

```
The Sun Also Rises
```

Этот метод использует довольно простой алгоритм: он не пытается различить важные и неважные слова и не обрабатывает аббревиатуры и апострофы.

Приведённый ниже код:

```
s = "what's happened to ted's IBM stock?"  
print(s.title())
```

ВЫВОДИТ:

```
What'S Happened To Ted'S Ibm Stock?
```

Метод lower()

Метод `lower()` возвращает копию строки `s`, в которой все символы имеют нижний регистр.

Приведённый ниже код:

```
s = 'F00 Bar 123 baz qUX'  
print(s.lower())
```

ВЫВОДИТ:

```
foo bar 123 baz qux
```

Метод upper()

Метод `upper()` возвращает копию строки `s`, в которой все символы имеют верхний регистр.

Приведённый ниже код:

```
s = 'F00 Bar 123 baz qUX'  
print(s.upper())
```

ВЫВОДИТ:

```
F00 BAR 123 BAZ QUX
```



Одно очень важное замечание о методах данной категории состоит в том, что они не изменяют исходную строку. Если вы хотите изменить строку `s`, нужно написать код: `s = s.lower()`. На самом деле тут вы создаёте совсем другой объект в памяти компьютера, просто он со старым названием `s`.

Примечание

Англо-русский словарь:

capitalize — писать прописными буквами, закрепить.

swarpcase — обменять регистр. swar — гл. обмениваться, case — случай, регистр, падеж, дело, расследование...

title — заголовок, титул.

lower — нижний.

upper — верхний.

Тема урока: строки

1. Методы `count()`
2. Метод `startswith()`
3. Метод `endswith()`
4. Метод `find()`
5. Метод `rfind()`
6. Метод `index()`
7. Метод `rindex()`
8. Метод `strip()`
9. Метод `lstrip()`
10. Метод `rstrip()`
11. Метод `replace()`

Аннотация. Строковый тип данных, основные методы поиска и замены.

Поиск и замена

Методы поиска и замены строк внутри других строк.

Каждый метод в этой группе поддерживает необязательные аргументы `<start>` и `<end>`. Как и в строковых срезах, действие метода ограничено частью исходной строки, начинающейся с позиции символа `<start>` и продолжающейся вплоть до позиции символа `<end>`, но не включающей её. Если параметр `<start>` указан, а параметр `<end>` нет, то метод применяется к части исходной строки от `<start>` до конца строки. Если параметры не заданы, то подразумевается, что `<start> = 0`, `<end> = len(s)`.

Метод count()

Метод `count(<sub>, <start>, <end>)` считает количество **непересекающихся** вхождений подстроки `<sub>` в исходную строку `s`.

Приведённый ниже код:

```
s = 'foo goo moo'
print(s.count('oo'))
print(s.count('oo', 0, 8)) # подсчет с 0 по 7 символ
```

ВЫВОДИТ:

```
3
2
```

Метод startswith()

Метод `startswith(<suffix>, <start>, <end>)` определяет, **начинается** ли исходная строка `s` подстрокой `<suffix>`. Метод возвращает значение `True`, если исходная строка начинается с подстроки `<suffix>`, или `False` в противном случае.

Приведённый ниже код:

```
s = 'foobar'
print(s.startswith('foo'))
print(s.startswith('baz'))
```

ВЫВОДИТ:

```
True
False
```

Метод endswith()

Метод `endswith(<suffix>, <start>, <end>)` определяет, **оканчивается** ли исходная строка `s` подстрокой `<suffix>`. Метод возвращает значение `True`, если исходная строка оканчивается на подстроку `<suffix>`, или `False` в противном случае.

Приведённый ниже код:

```
s = 'foobar'
print(s.endswith('bar'))
print(s.endswith('baz'))
```

ВЫВОДИТ:

```
True
False
```

Методы find(), rfind()

Метод `find(<sub>, <start>, <end>)` находит **индекс первого вхождения** подстроки `<sub>` в исходной строке `s`. Если строка `s` не содержит подстроки `<sub>`, то метод возвращает значение `-1`. Мы можем использовать данный метод наравне с оператором `in` для проверки: содержит ли заданная строка некоторую подстроку или нет.

Приведённый ниже код:

```
s = 'foo bar foo baz foo qux'
print(s.find('foo'))
print(s.find('bar'))
print(s.find('qu'))
print(s.find('python'))
```

ВЫВОДИТ:

```
0
4
20
-1
```

Метод `rfind(<sub>, <start>, <end>)` идентичен методу `find(<sub>, <start>, <end>)`, за тем исключением, что он ищет первое вхождение подстроки `<sub>`, начиная с конца строки `s`.

Методы index(), rindex()

Метод `index(<sub>, <start>, <end>)` идентичен методу `find(<sub>, <start>, <end>)`, за тем исключением, что он **вызывает ошибку** `ValueError: substring not found` во время выполнения программы, если подстрока `<sub>` не найдена.

Метод `rindex(<sub>, <start>, <end>)` идентичен методу `index(<sub>, <start>, <end>)`, за тем исключением, что он ищет первое вхождение подстроки `<sub>`, начиная с конца строки `s`.



Методы `find()` и `rfind()` являются более безопасными, чем `index()` и `rindex()`, так как не приводят к возникновению ошибки во время выполнения программы.

Метод strip()

Метод `strip()` возвращает копию строки `s`, у которой удалены все пробелы, стоящие **в начале и конце** строки.

Приведённый ниже код:

```
s = '    foo bar foo baz foo qux    '  
print(s.strip())
```

ВЫВОДИТ:

```
foo bar foo baz foo qux
```

Метод lstrip()

Метод `lstrip()` возвращает копию строки `s`, у которой удалены все пробелы, стоящие **в начале** строки.

Приведённый ниже код:

```
s = '    foo bar foo baz foo qux    '  
print(s.lstrip())
```

ВЫВОДИТ (символом `_` обозначены пробелы):

```
foo bar foo baz foo qux_____
```

Метод rstrip()

Метод `rstrip()` возвращает копию строки `s`, у которой удалены все пробелы, стоящие **в конце** строки.

Приведённый ниже код:

```
s = '    foo bar foo baz foo qux    '  
print(s.rstrip())
```

ВЫВОДИТ (символом `_` обозначены пробелы):

```
_____foo bar foo baz foo qux
```


Метод replace()

Метод `replace(<old>, <new>)` возвращает копию `s` со всеми вхождениями подстроки `<old>`, заменёнными на `<new>`.

Приведённый ниже код:

```
s = 'foo bar foo baz foo qux'
print(s.replace('foo', 'grault'))
```

выводит:

```
grault bar grault baz grault qux
```

Метод `replace()` может принимать необязательный третий аргумент `<count>`, который определяет количество замен.

Приведённый ниже код:

```
s = 'foo bar foo baz foo qux'
print(s.replace('foo', 'grault', 2))
```

выводит:

```
grault bar grault baz foo qux
```

Примечания

Примечание. Методы `strip()`, `lstrip()`, `rstrip()` могут принимать на вход необязательный аргумент `<chars>`. Необязательный аргумент `<chars>` – это строка, которая определяет набор символов для удаления.

Приведённый ниже код:

```
s = '-+-+abc+--+'
print(s.strip('+--'))
print(s.rstrip('+--'))
print(s.lstrip('+--'))
```

выводит:

```
abc
-+-+abc
abc+--+
```

Классификация символов

Методы в этой группе классифицируют строку на основе содержащихся в ней символов.

Метод `isalnum()`

Метод `isalnum()` определяет, состоит ли исходная строка из буквенно-цифровых символов. Метод возвращает значение `True`, если исходная строка является непустой и состоит **только** из буквенно-цифровых символов, или `False` в противном случае.

Приведённый ниже код:

```
s1 = 'abc123'  
s2 = 'abc$*123'  
s3 = ''  
  
print(s1.isalnum())  
print(s2.isalnum())  
print(s3.isalnum())
```

ВЫВОДИТ:

```
True  
False  
False
```

Обратите внимание, что метод `isalnum()` возвращает значение `True` даже в том случае, когда строка состоит только из буквенных или только из цифровых символов.

Приведённый ниже код:

```
s1 = 'BEEGEEK'  
s2 = '2202'  
  
print(s1.isalnum())  
print(s2.isalnum())
```

ВЫВОДИТ:

```
True  
True
```

Метод isalpha()

Метод `isalpha()` определяет, состоит ли исходная строка из буквенных символов. Метод возвращает значение `True`, если исходная строка является непустой и состоит **только** из буквенных символов, или `False` в противном случае.

Приведённый ниже код:

```
s1 = 'ABCabc'  
s2 = 'abc123'  
s3 = ''  
  
print(s1.isalpha())  
print(s2.isalpha())  
print(s3.isalpha())
```

ВЫВОДИТ:

```
True  
False  
False
```

Метод isdigit()

Метод `isdigit()` определяет, состоит ли исходная строка **только** из цифровых символов. Метод возвращает значение `True`, если исходная строка является непустой и состоит **только** из цифровых символов, или `False` в противном случае.

Приведённый ниже код:

```
s1 = '1234567'  
s2 = 'abc123'  
s3 = ''  
  
print(s1.isdigit())  
print(s2.isdigit())  
print(s3.isdigit())
```

ВЫВОДИТ:

```
True  
False  
False
```

Метод islower()

Метод `islower()` определяет, являются ли **все** буквенные символы исходной строки строчными (имеют нижний регистр). Метод возвращает значение `True`, если все буквенные символы исходной строки являются строчными, или `False` в противном случае.

Приведённый ниже код:

```
s1 = 'abc'
s2 = 'abc1$d'
s3 = 'Abc1$d'

print(s1.islower())
print(s2.islower())
print(s3.islower())
```

ВЫВОДИТ:

```
True
True
False
```

Обратите внимание, что метод `islower()` **игнорирует все небуквенные символы**.

Приведённый ниже код:

```
print('1234'.islower())
print('+-*/'.islower())
print('ab#%'.islower())
```

ВЫВОДИТ:

```
False
False
True
```

В первом и втором случаях у нас в строках отсутствуют буквенные символы в нижнем регистре, поэтому метод `islower()` и возвращает значение `False`.

Метод isupper()

Метод `isupper()` определяет, являются ли **все** буквенные символы исходной строки заглавными (имеют верхний регистр). Метод возвращает значение `True`, если все буквенные символы исходной строки являются заглавными, или `False` в противном случае.

Приведённый ниже код:

```
s1 = 'ABC'
s2 = 'ABC1$d'
s3 = 'Abc1$d'

print(s1.isupper())
print(s2.isupper())
print(s3.isupper())
```

ВЫВОДИТ:

```
True
True
False
```

Обратите внимание, что метод `isupper()` **игнорирует все небуквенные символы**.

Приведённый ниже код:

```
print('5678'.isupper())
print('!?_&'.isupper())
print('AB%$'.isupper())
```

ВЫВОДИТ:

```
False
False
True
```

В первом и втором случаях у нас в строках отсутствуют буквенные символы в верхнем регистре, поэтому метод `isupper()` и возвращает значение `False`.

Метод `isspace()`

Метод `isspace()` определяет, состоит ли исходная строка **только** из пробельных символов. Метод возвращает значение `True`, если строка состоит только из пробельных символов, или `False` в противном случае.

Приведённый ниже код:

```
s1 = '   '
s2 = 'abc1$d'

print(s1.isspace())
print(s2.isspace())
```

ВЫВОДИТ:

```
True
False
```

Для пустой строки метод `isspace()` также возвращает `False`, так как в этом случае строка не состоит из пробельных символов.

Приведённый ниже код:

```
s1 = ''
print(s1.isspace())
```

ВЫВОДИТ:

```
False
```

Тема урока: форматирование строк

1. Строковый метод `format()`
2. f-строки

Аннотация. Строковый тип данных, основные методы форматирования строк.

Метод `format()`

Хранить строки в переменных удобно, но часто бывает необходимо собирать строки из других объектов (строк, чисел и т.д.) и выполнять с ними нужные манипуляции. Для этой цели можно воспользоваться механизмом **форматирования строк**.

Рассмотрим следующий код:

```
birth_year = 1992
text = 'My name is Timur, I was born in ' + birth_year

print(text)
```

Такой код приводит к ошибке во время выполнения программы, поскольку мы пытаемся сложить число и строку:

```
TypeError: can only concatenate str (not "int") to str
```

Для решения такой проблемы мы можем использовать функцию `str()`, которая преобразует числовое значение в строку.

Приведенный ниже код:

```
birth_year = 1992
text = 'My name is Timur, I was born in ' + str(birth_year)

print(text)
```

ВЫВОДИТ:

```
My name is Timur, I was born in 1992
```

Такой код работает, но каждый раз преобразовывать число в строку не очень удобно. Для более наглядного форматирования мы можем использовать строковый метод `format()`.

Предыдущий код можно переписать в виде:

```
birth_year = 1992
text = 'My name is Timur, I was born in {}'.format(birth_year)

print(text)
```

Мы передаем необходимые параметры методу `format()`, а Python ставит их вместо фигурных скобок `{}` – **заполнителей**. Мы можем создавать сколько угодно заполнителей.

Приведенный ниже код:

```
birth_year = 1992
name = 'Timur'
profession = 'math teacher'
text = 'My name is {}, I was born in {}, I work as a {}.'.format(name,
birth_year, profession)

print(text)
```

ВЫВОДИТ:

```
My name is Timur, I was born in 1992, I work as a math teacher.
```

Для наглядности и гибкости форматирования мы можем использовать порядковый номер в заполнителе: `{0}`, `{1}`, `{2}` и т.д. Такой номер определяет позицию параметра, переданного методу `format()` (нумерация начинается с нуля):

```
birth_year = 1992
name = 'Timur'
profession = 'math teacher'
text = 'My name is {0}, I was born in {1}, I work as a
{2}.'.format(name, birth_year, profession)

print(text)
```

Параметр `name` встает в заполнителе `{0}`, параметр `birth_year` – в заполнителе `{1}` и т.д. Мы можем использовать одно и то же число в нескольких заполнителях или не использовать совсем, а также мы можем использовать числа в разном порядке.

Приведенный ниже код:

```
name = 'Timur'
city = 'Moscow'
text1 = 'My name is {0}-{0}-{0}!'.format(name, city)
text2 = '{1} is my city and {0} is my name!'.format(name, city)

print(text1)
print(text2)
```

ВЫВОДИТ:

```
My name is Timur-Timur-Timur!
Moscow is my city and Timur is my name!
```


f-строки

Метод `format()` хорошо справляется с задачей форматирования строк, однако если параметров много, то код может показаться немного избыточным.

Приведенный ниже код:

```
first_name = 'Taylor'
second_name = 'Swift'
country = 'USA'
birth_date = '1989/12/13'
birth_place = 'West Reading, Pennsylvania'
text = '{0} {1} is a very famous singer from the {2}. She was born on {3} in {4}.'.format(first_name, second_name, country, birth_date, birth_place)

print(text)
```

ВЫВОДИТ:

```
Taylor Swift is a very famous singer from the USA. She was born on 1989/12/13 in West Reading, Pennsylvania.
```

В Python 3.6 появилась новая разновидность строк — f-строки. Если поставить перед строкой префикс `f`, в заполнители можно будет включить код, например, имя переменной или любые другие выражения. f-строки обеспечивают чистый и интуитивно понятный способ форматирования строк.

Предыдущий код можно записать в виде:

```
first_name = 'Taylor'
last_name = 'Swift'
country = 'USA'
birth_date = '1989/12/13'
birth_place = 'West Reading, Pennsylvania'
text = f'{first_name} {last_name} is a very famous singer from the {country}. She was born on {birth_date} in {birth_place}.'

print(text)
```

На место заполнителя `{first_name}` встает значение переменной `first_name`, на место заполнителя `{last_name}` встает значение переменной `last_name` и т.д.

Как уже говорилось, помимо переменных в заполнителях f-строк мы можем использовать выражения.

Приведенный ниже код:

```
print(f'5 + 2 = {5 + 2}')
print(f'5 - 2 = {5 - 2}')
print(f'5 * 2 = {5 * 2}')
print(f'5 / 2 = {5 / 2}')
```

ВЫВОДИТ:

```
5 + 2 = 7
5 - 2 = 3
5 * 2 = 10
5 / 2 = 2.5
```

Примечания

Примечание 1. Почитать подробнее про форматирование строк можно в официальной документации по [ссылке](#). Отдельно можно почитать на английском языке про метод `format()` и f-строки по [ссылке](#) и по [ссылке](#).

Примечание 2. На русском языке можно почитать про форматирование строк по [ссылке](#) и по [ссылке](#).

Примечание 3. Про эволюцию форматирования в Python вы можете почитать в нашем официальном блоге на Хабре по [ссылке](#).

Примечание 4. До версии Python 3.12 повторное использование тех же кавычек, что и окружающие f-строку, вызывает ошибку синтаксиса.

Приведенный ниже код:

```
text = f'{'London'} is the capital of {'England'} and the {'United Kingdom'}'
```

приводит к возникновению ошибки:

```
SyntaxError: f-string: expecting '}'
```

В версии Python 3.12 это ограничение сняли. Обратите внимание, что на данный момент на Stepik **недоступна версия** Python 3.12, и если вы копируете код из своей IDE (в которой, возможно, уже стоит Python версии 3.12 и выше), пожалуйста, проверяйте, что вы не используете повторно одни кавычки.

Сравнение строк единичной длины

Начнем с примера сравнения строк, состоящих из одного символа. В Python это сравнение происходит путём сравнения кодов этих символов в таблице Unicode.

Приведённый ниже код:

```
print('a' > 'b')
print('a' < 'z')
```

ВЫВОДИТ:

```
False
True
```

Действительно, код символа `a` в таблице Unicode равен числу 97, а символа `b` – числу 98. Число 97 меньше числа 98, поэтому и символ `a` меньше символа `b`. Аналогично символ `z` больше символа `a`, потому что код символа `z` (число 122) больше кода символа `a` (число 97).

Предыдущий код полностью эквивалентен следующему коду:

```
print(ord('a') > ord('b'))
print(ord('a') < ord('z'))
```

Обратите внимание, что буквы в нижнем регистре всегда больше своих аналогов в верхнем регистре. Для букв русского алфавита это правило также работает.

Приведённый ниже код:

```
print('d' > 'D')
print('bl' < 'yl')
```

ВЫВОДИТ:

```
True
True
```

Ничего удивительного в этом нет, потому что буквы в нижнем регистре идут после букв в верхнем регистре в таблице Unicode.



Подробнее ознакомиться с таблицей символов Unicode можно по [ссылке](#). Обычно на практике достаточно оперировать [таблицей ASCII](#), которая является подмножеством таблицы Unicode. Первые 128 символов таблицы Unicode совпадают с таблицей ASCII.

Сравнение строк не единичной длины

Обычно мы работаем не с отдельными символами, а со строками, которые состоят из нескольких символов сразу.

Алгоритм сравнения строк:

1. Начинаем с первых символов каждой строки. Если символы равны, переходим к следующей паре символов
2. Когда находим первый отличающийся символ, строка с меньшим символом считается "меньше"
3. Если одна из строк заканчивается раньше, то более короткая считается "меньше"

Чтобы лучше разобраться с алгоритмом сравнения строк, рассмотрим несколько примеров.

Пример 1. Сравним строки `'hello'` и `'hell'`.

- Сравнение первых символов: `h` и `h` – оба символа равны, переходим к следующей паре символов
- Сравнение вторых символов: `e` и `e` – оба символа равны, переходим к следующей паре символов
- Сравнение третьих символов: `l` и `l` – оба символа равны, переходим к следующей паре символов
- Сравнение четвертых символов: `l` и `l` – оба символа равны, переходим к следующей паре символов
- Сравнение пятых символов: `o` (у первой строки) и отсутствующий символ (у второй строки) – вторая строка закончилась

Поскольку у второй строки закончились символы, а у первой строки они еще есть, считается, что первая строка больше второй. Поэтому `'hello' > 'hell'`.

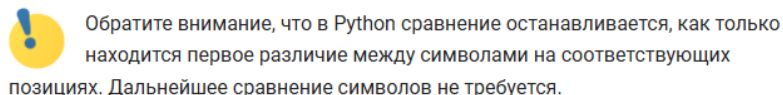
hello
hell

Пример 2. Сравним строки `'men'` и `'mya'`.

- 1) Сравнение первых символов: `m` и `m` – оба символа равны, переходим к следующей паре символов
- 2) Сравнение вторых символов: `e` (у первой строки) и `y` (у второй строки) – символ `e` (число 101) меньше символа `y` (число 121)

Так как строки начинают различаться со второго символа, то на основе этого символа и делается вывод о результатах сравнения строк. В данном случае символ `e` меньше символа `y`. Получаем, что `'men' < 'mya'`.

men
mya



Пример 3. Сравним строки 'Меееееooooooooow' и 'meow'.

1) Сравнение первых символов: М (у первой строки) и м (у второй строки) – символ М (число 77) меньше символа м (число 109).

Строки различаются уже на первых символах, и первый символ у первой строки меньше первого символа второй строки. Поэтому `'Meeeeeeeeooooow' < 'meow'`.

Meeeeeeooooow
meow

Примечания

Примечание 1. Нельзя путать сравнение чисел и сравнение строк, содержащих эти числа.

Приведённый ниже код:

```
print(10 > 9)
print('10' > '9')
```

ВЫВОДИТ:

True
False

Примечание 2. Мы можем сравнивать не только строки, состоящие из букв латинского алфавита, но и строки, состоящие из любых символов, которые входят в таблицу Unicode. Алгоритм сравнения строк при этом будет аналогичный – в соответствии с кодами символов в таблице Unicode.

Приведённый ниже код:

```
print('Тинькофф' == 'Т-банк')
print('¥' > '$£')
print(max('🐱', '🐤', '🐓'))
```

ВЫВОДИТ:

False
True


Примечание 3. В Python не поддерживается операция сравнения строк и чисел друг с другом.

Приведённый ниже код:

```
print('45' > 44)
```

ПРИВОДИТ К ВОЗНИКНОВЕНИЮ ОШИБКИ:

```
TypeError: '>' not supported between instances of 'str' and 'int'
```

Python пытается выполнить лексикографическое сравнение для строк и числовое сравнение для чисел. Но эти операции несопоставимы, и поэтому возникает ошибка.

Примечание 4. В Python встроенные функции `min()` и `max()` могут принимать строки в качестве аргументов и сравнивают их лексикографически (используя порядок символов в кодировке Unicode). Как несложно догадаться, функция `min()` вернёт самую "маленькую" строку, а `max()` – самую "большую" строку.

Приведённый ниже код:

```
print(max('tree', 'try', 'true'))  
print(min('cat', 'car', 'cape'))
```

выводит:

```
try  
cape
```

Обратите внимание, что мы не можем **одновременно** передавать строки и числа в качестве аргументов в функции `min()` и `max()`. Это является следствием того, что мы не можем сравнивать строки с числами.

Приведённый ниже код:

```
print(min('2', 8, '45', 90))
```

приводит к возникновению ошибки:

```
TypeError: '<' not supported between instances of 'int' and 'str'
```